



Building MCP Powered Agentic RAG Application: Step by Step Guide



Query



MCP Client



Contact MCP

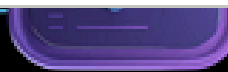
MCP Server



Freedium ^{beta}



Answer



Tool call



Vector DB search
MCP tool



Web search MCP
tool

Send context to MCP
client



To Data & Beyond

July 14, 2025

Building MCP-Powered Agentic RAG Application: Step-by-Step Guide (1/2)

A Step-by-Step Guide to Building an Agentic RAG Application with MCP



Youssef Hosni

9 min read

[Download article](#) 

Contents 8

1. Introduction to Model Context Protocol (MCP)
2. MCP-Powered Agentic RAG Architecture Overview
3. Setting Up the Working Environment
 - Dependencies and External Services
- 5 4. Set up an MCP server instance

[Show all 8 sections](#)

You can read the article for free from this [link](#)↗

While many tutorials show how to build Retrieval-Augmented Generation (RAG) systems by connecting a language model to a single database, this approach is often too limited.

This guide takes you to the next level with **Agentic RAG**, where you won't just build a system that retrieves information — you'll build one that intelligently decides *where* to look. It's the difference between an AI that can only read one assigned book and an AI that knows exactly which book to pull from the library.

To achieve this, we will use the **Model Context Protocol (MCP)**, a modern framework designed specifically for creating powerful, multi-tool AI agents. In this step-by-step tutorial, we will walk you through the entire process:

- Setting up a complete MCP server.
- Defining custom tools for both private data and live web searches.
- Constructing a full RAG pipeline that puts the agent in control.

By the end, you'll have a fully functional and intelligent application capable of fielding complex queries by dynamically sourcing the best possible context for a truly accurate answer.

Building MCP Powered Agentic RAG Application: Step by Step Guide

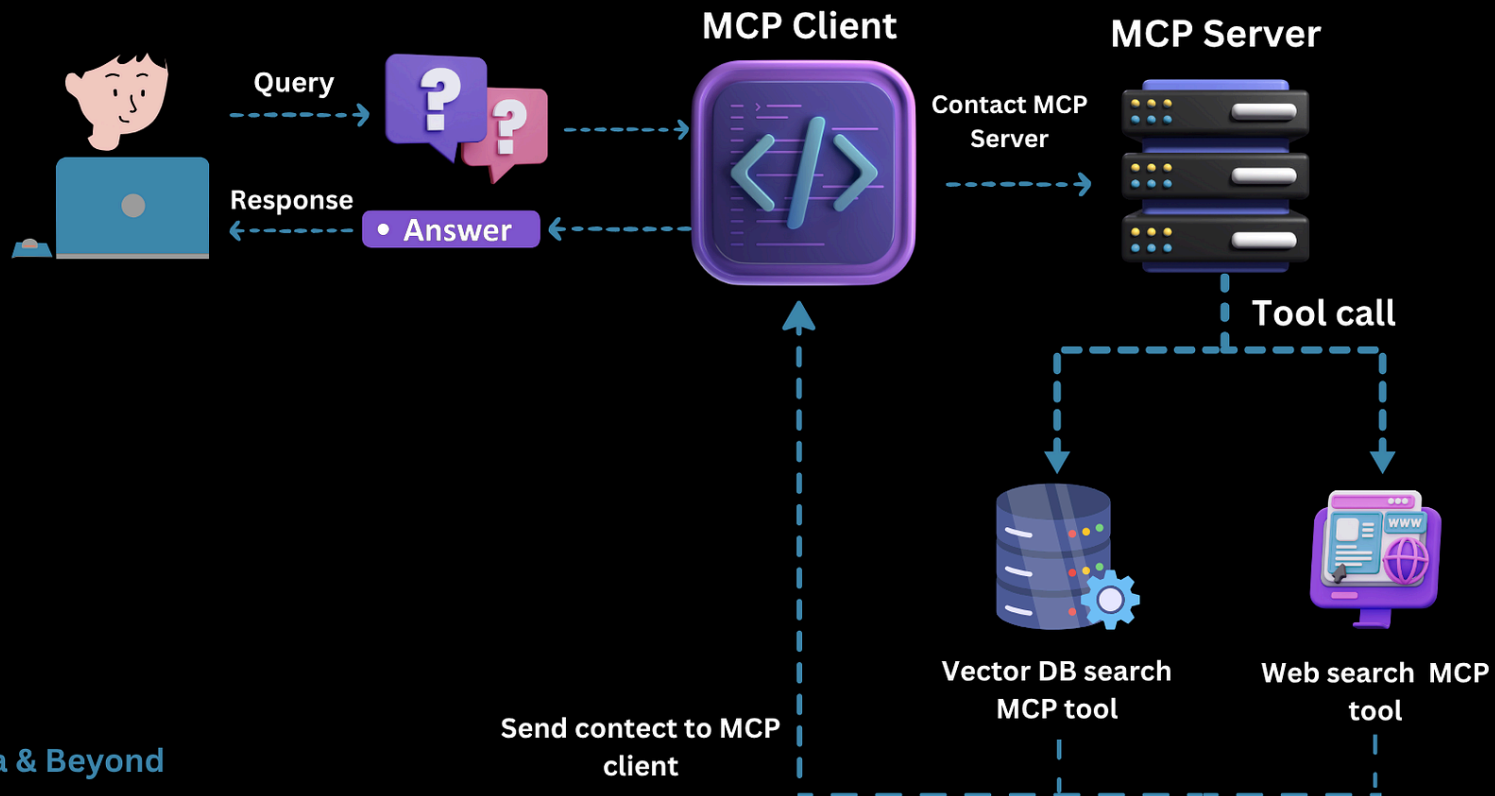


Table of Contents:

1. Introduction to Model Context Protocol (MCP)
2. MCP-Powered Agentic RAG Architecture Overview
3. Setting Up the Working Environment

4. Set up an MCP server instance

5. [Define MCP Tools \[Part 2\]](#) 

6. [Building RAG Pipeline \[Part 2\]](#) 

7. [Putting Everything Together and Testing the System \[Part 2\]](#) 

You can find all the codes in this [GitHub Repository](#) 

Most insights I share in Medium have previously been shared in my weekly newsletter, To Data & Beyond.

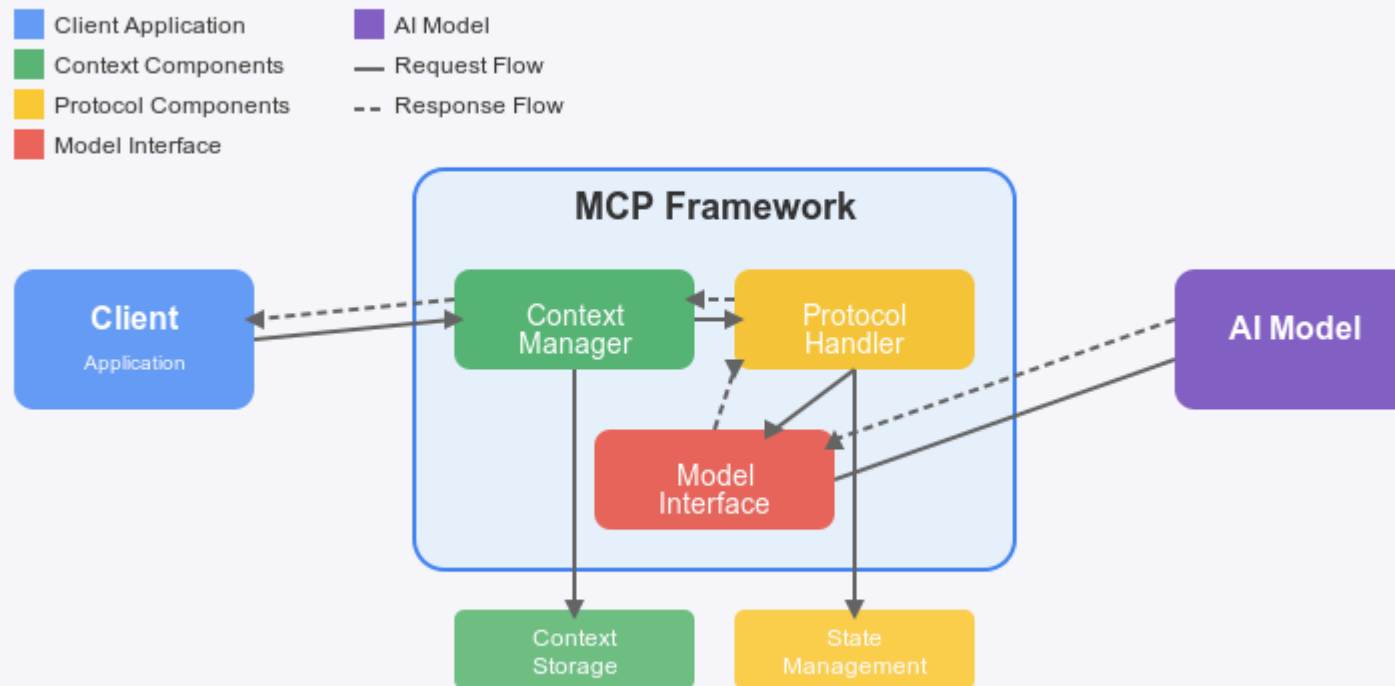
If you want to be up-to-date with the frenetic world of AI while also feeling inspired to take action or, at the very least, to be well-prepared for the future ahead of us, this is for you.

 **Subscribe below**  *to become an AI leader among your peers and receive content not present in any other platform, including Medium:*

[To Data & Beyond](#) | [Youssef Hosni](#) | [Substack](#)  *Data Science, Machine Learning, AI, and what is beyond them. Click to read To Data & Beyond, by Youssef Hosni, a...*

1. Introduction to Model Context Protocol (MCP)

Model Context Protocol (MCP) Working Process



MCP, or Model Context Protocol, is an open standard that enables large language models (LLMs) to securely access and utilize external data, tools, and services.

Think of it as a universal connector, like USB-C for AI, allowing different AI applications to interact with various data sources and functionalities. It essentially standardizes how AI models receive and use context, such as files, prompts, or callable functions, to perform tasks more effectively.

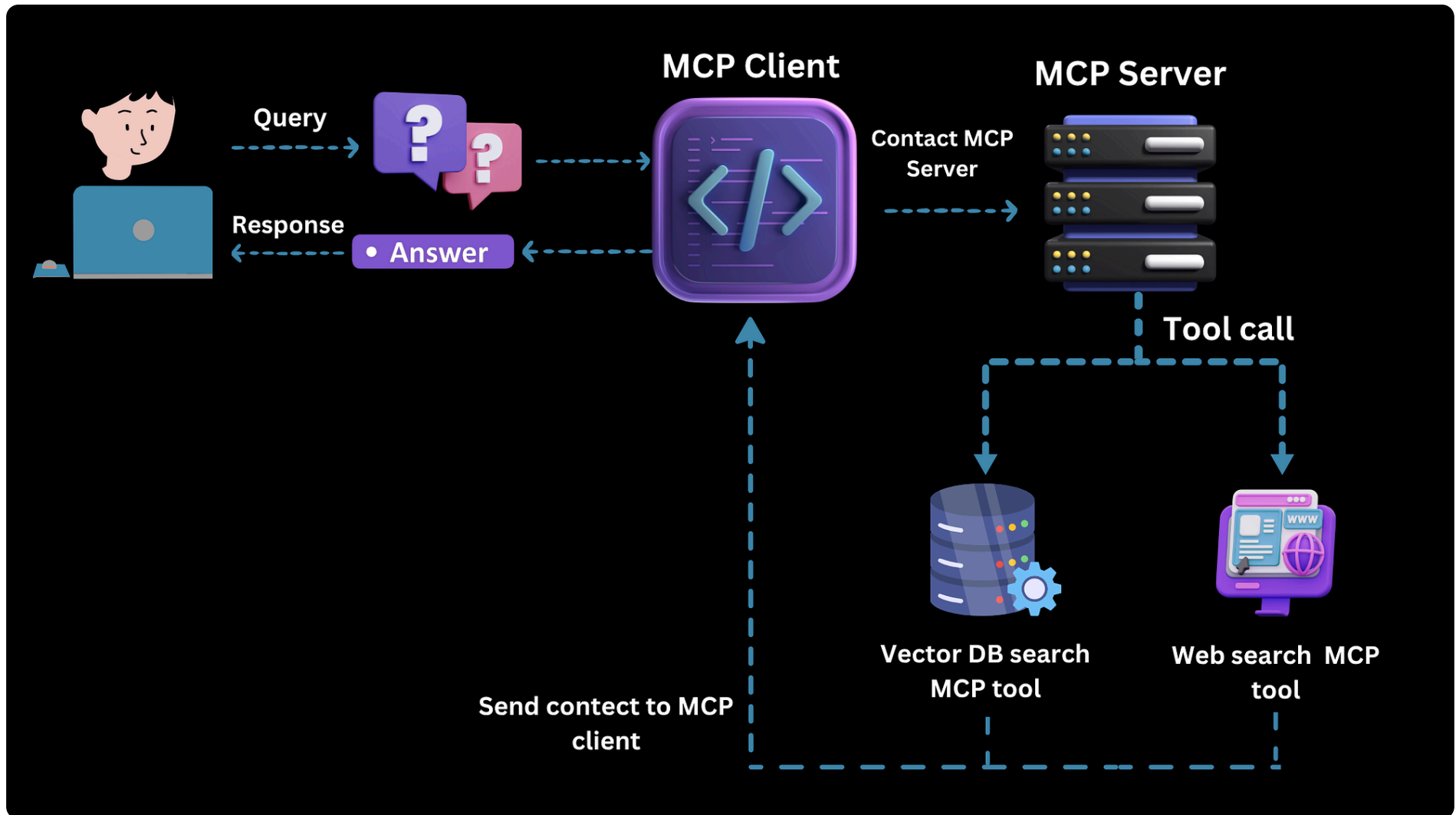
- **Standardized Connection:** MCP provides a consistent way for AI models to interact with external systems, regardless of the specific AI model or the underlying technology.
- **Secure Access:** MCP ensures secure access to resources by defining clear boundaries and permissions for how AI models can interact with external data and tools.
- **Two-Way Communication:** MCP facilitates both sending information to the AI model (context) and receiving results or actions back from the model, enabling dynamic and interactive AI applications.
- **Ecosystem of Tools:** MCP is designed to foster a vibrant ecosystem of "MCP servers," which are applications or services that expose their functionality or data through the MCP standard, and "MCP clients," which are the AI applications that connect to these servers.

[Everything I've Written, One Button Away, With 40% Off](#)  *Announcing My Ebooks Bundle + 50% Discount to my Followers*

2. MCP-Powered Agentic RAG Architecture Overview

Before we roll up our sleeves and dive into the code, it's crucial to understand the high-level architecture of our MCP-powered Agentic RAG application. This system is designed to be intelligent, resourceful, and responsive.

The diagram below illustrates the complete workflow, from the user's initial question to the final, context-enriched answer.



The architecture is composed of several key components that work in concert:

1. **The User:** The starting point of the entire process. A user has a question and interacts with the system via a front-end application (e.g., a web interface on their laptop).
2. **MCP Client:** This is the user-facing part of our application. It's responsible for receiving the user's Query, communicating with the backend server, and ultimately presenting the final Response back to the user.

3. **MCP Server (The Agent):** This is the core of our system — the "brain" of the operation. When the MCP Server receives a request, it doesn't just try to answer from its own pre-trained knowledge. Instead, it acts as an **agent**, intelligently deciding if it needs more information to provide a high-quality answer.
4. **Tools:** These are external resources the MCP Server can use to find information. Our architecture features two powerful tools:
 - **Vector DB search MCP tool:** This tool connects to a specialized vector database. This database is ideal for storing and searching through your private documents, internal knowledge bases, or any custom text corpus. It excels at finding semantically similar information (the "Retrieval" in RAG).
 - **Web search MCP tool:** When the answer isn't in our private data, this tool allows the agent to search the live internet for the most current and relevant public information.

Now, let's trace the path of a request through the system:

1. **Query Submission:** The user types a question ("Query") into the application.
2. **Client to Server Communication:** The MCP Client receives the query and forwards it to the MCP Server for processing.
3. **The Agentic Decision (Tool Call):** This is where the magic happens. The MCP Server analyzes the query and determines the best course of action. It makes a "Tool call," deciding whether it should:
 - Query the Vector DB for internal, proprietary knowledge.
 - Perform a Web search for real-time, public information.
 - Use a combination of both, or none at all if the question is general enough.
4. **Information Retrieval (The "R" in RAG):** The selected tool(s) execute the search and retrieve relevant documents, articles, or data snippets. This retrieved information is the "context."

5. Context Augmentation: The retrieved context is sent back and used to augment the original request. The system now has both the user's original query and a rich set of relevant information.

6. Response Generation: The MCP Client, now armed with this powerful context, generates a comprehensive and accurate answer. This process is known as "Retrieval-Augmented Generation" (RAG) because the final response is generated based on the retrieved information.

7. Final Answer: The final, context-aware Response is sent back to the user, providing an answer that is not just plausible, but well-supported by real data.

By orchestrating these components, we create a system that can answer complex questions by intelligently seeking out and synthesizing information from multiple sources. Now that we understand the "what" and "why," let's move on to the "how."

3. Setting Up the Working Environment

With the architecture mapped out, it's time to get our hands dirty and set up the foundation for our application. In this section, we'll configure our project, install the necessary libraries, and launch the external services our agent will rely on.

Before you begin, make sure you have the following installed on your system:

- **Python 3.8+** and **pip**
- **Docker** (for running our vector database)

Let's start by organizing our files. A clean structure makes the project easier to manage. Create a root folder for your project (e.g., `mcp-agentic-rag`) and create the following files inside it. We'll populate them as we go.

```
1 /mcp-agentic-rag
2 |-- .env
```

```
3 |-- mcp_server.py          # Our main application and server logic
4 |-- rag_app.py             # Our RAG-specific code and data
```

Now, open your **mcp_server.py** file and add the following code. This script will handle imports, load environment variables, and define the core configuration constants for our service.

```
1 import os
2 from typing import List
3 import requests
4
5 from dotenv import load_dotenv
6 from mcp.server.fastmcp import FastMCP
7
8 # Updated import: Assumes FAQEngine and the FAQ text are in 'rag_app.py'
9 from rag_app import FAQEngine, PYTHON_FAQ_TEXT
10
11 # Load environment variables from .env file
12 load_dotenv()
13
14 # Configuration constants
15 QDRANT_URL = "http://localhost:6333"
16 COLLECTION_NAME = "python_faq_collection" # Using a new collection for the Python data
17 HOST = "127.0.0.1"
18 PORT = 8080
```

We import standard libraries like `os` and `requests`, but the key imports are:

- `load_dotenv` from `python-dotenv`: A handy utility to load configuration (like API keys) from a separate `.env` file, keeping our code clean and secure.
- `FastMCP` from `mcp.server.fastmcp`: The core class from the MCP framework that we'll use to build our server.
- `FAQEngine` and `PYTHON_FAQ_TEXT` from `rag_app`: These are custom components we'll build shortly. `PYTHON_FAQ_TEXT` will hold our raw data, and `FAQEngine` will be the class that handles embedding and storing it.

We will also define easily accessible variables for our Qdrant connection and server address. Using constants makes the code more readable and easier to modify later.

Dependencies and External Services

Our agent can't work in a vacuum. It needs its tools and libraries.

1. Install Python Libraries

Open your terminal in the project directory and install the required Python packages using `pip`:

```
1 pip install mcp-server qdrant-client python-dotenv requests
```

- **mcp-server**: The core library for our MCP agent.
- **qdrant-client**: The official Python client to interact with the Qdrant vector database.
- **python-dotenv**: For loading our `.env` file.
- **requests**: A standard library for making HTTP requests, which our web search tool will use.

2. Create the `.env` File

Create a file named `.env` in your root directory. For now, it can be empty. This file is where you would store sensitive information like API keys if your tools required them (e.g., an `OPENAI_API_KEY` or a `FIRECRAWL_API_KEY`).

```
1 # .env file
2 # Add any API keys or secrets here later
3 OPENAI_API_KEY="sk- ..."
4 FIRECRAWL_API_KEY = "Your API Key"
```

3. Launch the Vector Database (Qdrant)

Our Vector DB search MCP tool needs a database to talk to. We'll use Docker to quickly and easily run an instance of Qdrant. Execute the following command in your terminal:

```
1 docker run -p 6333:6333 -p 6334:6334 \
2     -v $(pwd)/qdrant_storage:/qdrant/storage \
3     qdrant/qdrant
```

This command does three things:

- Downloads the official Qdrant image if you don't have it.
- Starts a container and maps two ports: 6333 for the API (which our app will use) and 6334 for the web UI.
- Mounts a local directory (`qdrant_storage`) to persist the database data on your host machine.

You can verify it's running by opening <http://localhost:6334/> in your browser. You should see the Qdrant dashboard.

Great! Our foundation is laid. We have our project structure, initial configuration, all necessary libraries installed, and a running vector database. In the next section, we'll set up the MCP server instance.

Everything I've Written, One Button Away, With 40% Off [🔗](#) *Announcing My Ebooks Bundle + 50% Discount to my Followers*

4. Set up an MCP server instance

With our environment configured, the next logical step is to create the central component of our application: the MCP server instance. This object, an instance of the FastMCP class, will act as the orchestrator for our agent. It's the "brain" that will manage incoming requests, know which tools are available, and handle the overall lifecycle of a query.

```
1 # Create an MCP server instance
2 mcp_server = FastMCP("MCP-RAG-app",
3                       host=HOST,
4                       port=PORT,
5                       timeout=30)
```

Let's quickly break down the parameters we passed to the FastMCP constructor:

- **"MCP-RAG-app"**: This is a unique name for our application. It's used internally by the MCP framework for identification and logging, making it easier to distinguish between different MCP services if you're running multiple.
- **host=HOST**: This parameter tells the server which network address to listen on. We're using the HOST constant we defined earlier ("127.0.0.1"), which means the server will only be accessible from the local machine (localhost). This is a secure default for development.
- **port=PORT**: This specifies the network port for the server. We've set it to 8080. Once running, our application will be available at <http://127.0.0.1:8080>. [🔗](#)

- **timeout=30:** This is an important setting for reliability. It sets a global timeout of 30 seconds for requests. If a tool call takes longer than this, the server will stop waiting and return a timeout error, preventing the application from hanging indefinitely.

Our server is now instantiated in the code, but it's like a brain without senses. It's ready to run, but it doesn't know how to *do* anything useful yet. In the next steps, we'll give it its "senses" by defining the tools it can use to retrieve information from the vector database and the web.

If you like the article and would like to support me, make sure to:

- 🖐️ Clap for the story (50 claps) to help this article be featured
- Subscribe to [the Data & Beyond](#) ↗ Newsletter
- Follow me on [Medium](#) ↗
- 📖 View more content on my [Medium profile](#) ↗
- 🔔 Follow Me: [LinkedIn](#) ↗ | [Youtube](#) ↗ | [GitHub](#) ↗ | [Twitter](#) ↗

Subscribe to my newsletter, To Data & Beyond, to get full and early access to my articles:

[To Data & Beyond](#) | [Youssef Hosni](#) | [Substack](#) ↗ *Data Science, Machine Learning, AI, and what is beyond them. Click to read To Data & Beyond, by Youssef Hosni, a...*

Are you looking to start a career in data science and AI, and do not know how? I offer data science mentoring sessions and long-term career mentoring:

- Mentoring sessions: <https://lnkd.in/dXeg3KPW> ↗
- Long-term mentoring: <https://lnkd.in/dtdUYBrM> ↗

Let's have a 1:1 call



- 1:1 Free Mentorship
- Data Science Career Planning
- Data Science Project Review

 topmate.io/youssef_hosni



By **Freedium.** – *made with care.*

[About](#)

[Privacy](#)

[Terms](#)

[RSS](#)

[GitHub](#)

[Codeberg](#)

